

Paddle fluid 分布式训练



0. 前言

Infinite后期需要支持多机GPU分布式训练，本次调研了Paddle fluid的分布式训练。主要涉及到单机多卡、多机多卡训练，Pserver模式的分布式训练、NCCL2模式的分布式训练。

由于在使用Paddle fluid v0.14.0进行分布式训练时存在[异常退出的bug](#)，目前使用的Paddle版本是Paddle fluid v1.0.1。

1. 概述

PaddlePaddle 是公司开源的深度学习平台，目前最新版本是v1.2.0。PaddlePaddle的核心架构如下图所示，主要包含训练框架和预测框架。

第一层是API接口，采用Python语言编写，大致分为两大类：

一类是组网类API，包括控制逻辑、构成逻辑、计算逻辑、IO模块、读取数据模块等；

另一类是执行类底层API，把用户组好的模型进行分析，并将计算方法进行合并优化，以此提高训练速度。

组网类API： 包含通用、控制、计算、优化、IO等类型的API

(1) Variables：PaddlePaddle中的变量，可以是Tensor、Parameter，或RPCClient。概念类似高级语言中的变量，有不同类型。

(2) Layers：PaddlePaddle中用户配置模型的基础模块，Layer表示一个或者一组紧密关联的计算，Layers可以通过输入输出连接起来。

(3) Block：Block表示一组连续的计算逻辑，通常是一个或多个顺序Layer组成。通常主模型是一个Block0。另外在Control Flow中，比如While、Ifelse，也会单独形成一个子Block。

(4) Control Flow：PaddlePaddle支持If-else、While、Switch等编程语言中常见的Control Flow，以确保模型的灵活表达。Control Flow通常以Block的形式存在。

(5) Program：包含了1个或者多个Block，表示一个完整的模型执行单元。执行器需要完整的执行Program，并保证读写关系符合用户的预期。

执行类API：

(1) 简单的单卡顺序执行器



Python | 收起 ^

1 Executor

(2) 多卡多线程数据并行执行器

</>

Python | 收起 ^

```
1 ParallelExecutor
2 exe = fluid.ParallelExecutor(
3     use_cuda,
4     loss_name=loss.name
5 )
```

(3) 生成分布式的中间表达

</>

Python | 收起 ^

```
1 t = distribute_transpiler.DistributeTranspiler()
2 t.transpile(trainer_id, pservers=pserver_endpoints, trainers=trainers)
```

[Paddle fluid python API参考](#)

2. Paddle fluid分布式设计模式

2.1 分布式设计思想

深度学习分布式训练主要分为两种：模型并行、数据并行。



在模型并行方式下，模型的层和参数将被分布在多个节点上，模型在一个mini-batch的前向和反向训练中，将经过多次跨节点之间的通信。每个节点只保存整个模型的一部分。

在数据并行方式下，每个节点保存有完整的模型的层和参数，每个节点独立完成前向和反向计算，然后完成梯度的聚合并同步的更新所有节点上的参数。

Fluid当前版本仅提供数据并行方式。

2.2 分布式训练模式

在数据并行模式的训练中，Fluid使用了两种通信模式，用于应对不同训练任务对分布式训练的要求，分别为RPC通信和Collective通信。其中RPC通信方式使用gRPC，Collective通信方式使用NCCL2。

2.2.1 Pserver模式分布式训练原理

Pserver模式是一种基于RPC通信的数据并行分布式训练方式。Pserver模式分布式训练会启动多个pserver进程和多个trainer进程，每个pserver进程会保存一部分模型参数，并负责接收从trainer发送的梯度并更新这些模型参数；每个trainer进程会保存一份完整的模型，并使用一部分数据进行训练，然

后向pserver发送梯度，最后从pserver拉取更新后的参数。



pserver进程可以在和trainer完全不同的计算节点上，也可以和trainer公用节点。一个分布式任务所需要的pserver进程个数通常需要根据实际情况调整，以达到最佳的性能，然而通常来说pserver的进程不会比trainer更多。在使用GPU训练时，pserver可以选择使用GPU或只使用CPU，如果pserver也使用GPU，则会增加一次从CPU拷贝接收到的梯度数据到GPU的开销，在某些情况下会导致整体训练性能降低。

2.2.2 NCCL2模式分布式训练

NCCL2的简单介绍。使用NCCL2进行分布式训练，是不需要启动pserver进程的，每个trainer进程都保存一份完整的模型参数，在完成计算梯度之后通过trainer之间的相互通信，Reduce梯度数据到所有节点的所有设备然后每个节点在各自完成参数更新。



目前使用NCCL2进行分布式训练仅支持同步训练方式。使用NCCL2方式的分布式训练，更适合模型体积较大，并需要使用同步训练和GPU训练，如果硬件设备支持RDMA和GPU Direct，可以达到很高的分布式训练性能。

3. Paddle fluid单机训练

单机训练采用启动单个docker容器的方式，将宿主机上的训练代码和数据挂载到容器中，然后进行训练。

3.1 单机单卡训练

在Docker中使用GPU进行训练有2种方式：手动将GPU设备挂载到容器内部、使用Nvidia的nvidia-docker。

3.1.1 手动挂载GPU设备到容器内部

(1) 启动容器

在启动容器时，将相关的CUDA库和GPU设备挂载到容器内部。下列脚本操作将CUDA库、GPU设备、主机上的训练代码和数据挂载到docker容器内部。

</>

Shell | 收起 ^

```
1 export CUDA_SO="$(\ls /usr/lib/x86_64-linux-gnu/libcuda* | xargs -I{} echo '-v {}:{}'.)' $(\ls /usr/lib/x86_64-linux-gnu/libnvidia* | xargs -I{} echo '-v {}:{}'.)'
2 export DEVICES=$(\ls /dev/nvidia* | xargs -I{} echo '--device {}:{}'.)'
3 docker run -it --rm ${CUDA_SO} ${DEVICES} -v /root/workspace/:/opt
  hub.baidubce.com/paddle-v1.0.1-gpu:latest /bin/bash
```

注意：不同机器上CUDA库的位置不同，在实际中需要做相应的调整，本次实验机器上CUDA库的位置于/usr/lib/x86_64-linux-gnu目录下。上面的docker镜像名需要设置为自己的镜像名。

(2) 设置环境变量

启动容器之后，运行调用了GPU的训练脚本时，容易出现一个问题：`cudaGetDeviceCount failed in GetCUDADeviceCount: no CUDA-capable device is detected`。原因是容器中的CUDA库版本和CUDA驱动不匹配，由于已经将宿主机上的CUDA库挂载到容器内部，设置LD_LIBRARY_PATH环境变量指定CUDA库的位置就可以。脚本如下：

</>

Shell | 收起 ^

```
1 export LD_LIBRARY_PATH=/usr/lib/x86_64-linux-gnu/:$LD_LIBRARY_PATH
```

在使用单机单卡训练时，如果Docker挂载了多张GPU卡，训练脚本会占用多张卡，但是只使用其中一张卡进行训练，可以使用 `CUDA_VISIBLE_DEVICES` 环境变量指定特定的GPU。

</>

Shell | 收起 ^

```
1 export CUDA_VISIBLE_DEVICES=0
```

PaddleCloud的trainer.py训练脚本在使用GPU时，需要额外设置一些环境变量。

</>

Shell | 收起 ^

```
1 export TRAINING_ROLE=TRAINER
2 export PADDLE_USE_GPU=1
3 export PADDLE_IS_LOCAL=1
```

3.1.2 使用nvidia-docker

(1) 启动容器

在启动容器时，nvidia-docker会自动获取主机硬件资源，如GPU显卡资源。如果docker镜像中的CUDA库和宿主机上的GPU驱动不匹配，需要将宿主机上的CUDA库挂载到容器内。由于测试镜像中的CUDA库比较老，需要将宿主机上的CUDA库映射到容器内部。使用nvidia-docker启动docker容器的命令如下：

</>

Shell | 收起 ^

```
1 export CUDA_SO="$(\ls /usr/lib/x86_64-linux-gnu/libcuda* | xargs -I{} echo '-v {}:{}'.)' $(\ls /usr/lib/x86_64-linux-gnu/libnvidia* | xargs -I{} echo '-v {}:{}'.)"
2 nvidia-docker run -it --privileged=true --rm --name=nvidia-docker-test ${CUDA_SO} -v /root/workspace/:/opt hub.baidubce.com/paddle-v1.0.1-gpu:latest /bin/bash
```

(2) 设置环境变量

后续的配置，参考手动挂载GPU设备到容器内部

3.2 单机多卡训练

在单机单卡训练的基础上，可以使用多张卡训练。使用Paddle fluid的ParallelExecutor接口进行多张卡的并行计算。

(1) 参数随机初始化

在使用多张GPU卡进行训练时，参数需要先在GPU0上初始化，再经由fluid.ParallelExecutor分发到多张显卡上，即下面的步骤不可少。

```
</> Python | 收起 ^  
  
1 exe = fluid.Executor(fluid.CUDAPlace(0))  
2 exe.run(program=fluid.default_startup_program())
```

(2) 多卡训练

使用fluid.ParallelExecutor运行训练fluid.Program，核心代码如下：

```
</> Python | 收起 ^  
  
1 # set ParallelExecutor  
2 train_exe = fluid.ParallelExecutor(use_cuda, loss_name=avg_cost.name,  
   main_program=main_program)  
3 test_exe = fluid.ParallelExecutor(use_cuda, main_program=test_program,  
   share_vars_from=train_exe)  
4  
5 # call run function  
6 train_exe.run(fetch_list=[avg_cost.name], feed=feeder.feed(data))  
7 loss_t, acc_t = test_exe.run(fetch_list=[avg_cost.name, acc.name],  
   feed=feeder.feed(test_data))
```

注意事项：

1. ParallelExecutor 的构造函数需要指明要执行的 fluid.Program，并在执行过程中不能修改。默认值是 fluid.default_main_program()
2. ParallelExecutor 需要明确指定是否使用 CUDA 显卡进行训练。在显卡训练模式下会占用全部显卡。用户可以配置 CUDA_VISIBLE_DEVICES 来修改占用的显卡。
3. ParallelExecutor 的 run 函数和 Executor 的 run 函数不同，ParallelExecutor 的 fetch_list 传入的是各种对象实例的名称，其返回结果是list。比如上面代码中的loss_t是一个list，如果有2张卡，则每次训练之后loss_t值是长度为2的list。在计算loss的时候，取loss_t的平均值就可以。

3.3 保存checkpoint用于恢复训练

在训练的过程中，可能存在一些突发情况导致训练中止。可以在一些节点上将当前的训练状态保存下来，以便在将来需要的时候恢复训练环境继续进行训练，这一般被称为"checkpoint"。可以使用 `fluid.io.save_checkpoint` 接口保存checkpoint

</>

Python | 收起 ^

```
1 import paddle.fluid as fluid
2
3 exe = fluid.Executor(fluid.CPUPlace())
4 path = "./checkpoints"
5 prog = fluid.default_main_program()
6 trainer_args = {"epoch_id": 200,
7                 "step_id": 20} # just an example
8 # 保存checkpoint
9 fluid.io.save_checkpoint(executor=exe,
10                          checkpoint_dir=path,
11                          trainer_id=0,
12                          trainer_args=trainer_args,
13                          main_program=prog,
14                          max_num_checkpoints=3)
15
16
17 # 载入checkpoint用于恢复训练
18 fluid.io.load_checkpoint(executor=exe, checkpoint_dir=path,
19                          serial=9, main_program=prog)
```

通过调用 `fluid.io.save_checkpoint` 函数，PaddlePaddle fluid会对默认fluid.Program也就是prog中的所有模型变量进行扫描，根据一系列内置的规则自动筛选出其中所有需要保存的变量，并将他们保存到指定的path目录下。`fluid.io.save_checkpoint` 的各个参数中，`trainer_id`在单机情况下设置为0即可；`trainer_args` 为一个Python dict，用于给定当前的epoch_id和step_id；`max_num_checkpoints` 用于表示的最大checkpoint数量，如果目录中已经存在的checkpoint数量超过这个值，那最早的checkpoint将被删除。

通过调用 `fluid.io.load_checkpoint` 函数，PaddlePaddle fluid会对 prog 中的所有模型变量进行扫描，根据内置规则自动筛选出需要加载的变量，并尝试从 path 之中加载它们。

参数 `serial` 用来标记具体要加载的checkpoint的版本号。在保存checkpoint的时候，一个checkpoint会被保存在一个子目录中，并在目录名上体现出自己的版本号。一般越大的版本号表示这个checkpoint越新。

4. Paddle fluid分布式训练

Paddle fluid的分布式训练有两种模式，即Pserver模式和NCCL2模式。

4.1 Pserver模式的分布式训练

Pserver模式分布式训练一般使用docker方式进行部署，本次实验启动3个docker，其中包含2个trainer docker，1个pserver docker。

(1) 多机单卡分布式训练

本次实验参考了paddlecloud的demo，因此，在启动容器时，需要设置环境变量：
在pserver容器中设置如下环境变量：

</>

Python | 收起 ^

```
1 export TRAINING_ROLE=PSERVER
2 export PADDLE_PORT=6174
3 export PADDLE_PSERVERS=172.17.0.2
4 export PADDLE_TRAINERS_NUM=2
5 export POD_IP=172.17.0.2
6 export LD_LIBRARY_PATH=/usr/lib/x86_64-linux-gnu/:$LD_LIBRARY_PATH
```

在trainer容器中设置如下环境变量：

</>

Python | 收起 ^

```
1 export PADDLE_USE_GPU=1
2 export PADDLE_PORT=6174
3 export PADDLE_PSERVERS=172.17.0.2
4 export TRAINING_ROLE=TRAINER
5 export PADDLE_TRAINERS_NUM=2
6 export PADDLE_TRAINER_ID=1
7 export POD_IP=172.17.0.4
8 export CUDA_VISIBLE_DEVICES=1
9 export LD_LIBRARY_PATH=/usr/lib/x86_64-linux-gnu/:$LD_LIBRARY_PATH
```

其中，CUDA_VISIBLE_DEVICES环境变量为每个训练任务指定一张GPU卡

(2) 多机多卡分布式训练

在多机单卡训练的基础上，为每个训练任务分配2张GPU卡，并划分好数据，即可进行多机多卡。相关代码如下：

</>

Python | 收起 ^

```
1 if is_local:
2     train_loop(fluid.default_main_program(), test_program)
3 else:
4     port = os.getenv("PADDLE_PORT", "6174")
5     pserver_ips = os.getenv("PADDLE_PSERVERS") # ip,ip...
6     eplist = []
7     for ip in pserver_ips.split(","):
```

```

8     eplist.append(':'.join([ip, port]))
9     pserver_endpoints = ','.join(eplist) # ip:port,ip:port...
10    trainers = int(os.getenv("PADDLE_TRAINERS_NUM", "0"))
11    current_endpoint = os.getenv("POD_IP") + ":" + port
12    trainer_id = int(os.getenv("PADDLE_TRAINER_ID", "0"))
13    t = fluid.DistributeTranspiler()
14    t.transpile(trainer_id, pservers=pserver_endpoints, trainers=trainers)
15    if training_role == "PSERVER":
16        pserver_prog = t.get_pserver_program(current_endpoint)
17        pserver_startup = t.get_startup_program(current_endpoint,
18                                                pserver_prog)
19        exe.run(pserver_startup)
20        exe.run(pserver_prog)
21    elif training_role == "TRAINER":
22        train_loop(t.get_trainer_program(), test_program)
23    exe.close() # exit

```

注意：在Paddle fluid分布式训练中，一个节点完成训练数据后退出，其他节点都会一直等待，僵死直到被kill。可以在train_loop之后，调用exe.close()退出。

在实验中，多机多卡训练由于docker不再同一台机器上，每轮的训练时间有点长，估计时间耗费在网络传输上。

4.2 NCCL2模式的分布式训练

NCCL2模式的分布式训练，由于没有parameter server角色，trainer之间互相通信。本次实验启动2个docker容器，NCCL2模式的分布式训练核心代码如下：

(1) 核心代码

</>
Python | 收起 ^

```

1 # get trainer id from env
2 trainer_id = int(os.getenv("PADDLE_TRAINER_ID", "0"))
3 # get port
4 port = os.getenv("PADDLE_PORT", "6174")
5 # get paddle trainer ips
6 worker_ips = os.getenv("PADDLE_TRAINER_IPS") # ip,ip...
7 worker_endpoints = ""
8 for ip in worker_ips.split(","):
9     worker_endpoints += ':'.join([ip, port]) + ","
10 worker_endpoints = worker_endpoints[0:-1]
11 print ("worker endpoints: {0}".format(worker_endpoints))
12 trainer_nums = 2
13 current_endpoint = os.getenv("POD_IP") + ":" + port
14

```

```
15 config = fluid.DistributeTranspilerConfig()
16 config.mode = "nccl2"
17 t = fluid.DistributeTranspiler(config=config)
18 t.transpile(trainer_id, trainers=worker_endpoints,
    current_endpoint=current_endpoint)
19 startup_exe = fluid.Executor(fluid.CUDAPlace(0))
20 startup_exe.run(fluid.default_startup_program())
21
22 exe = fluid.ParallelExecutor(use_cuda,
23     loss_name=avg_cost.name,
24     num_trainers=trainer_nums, trainer_id=trainer_id)
25
26 # parallel train and test
27 train_exe = fluid.ParallelExecutor(use_cuda, loss_name=avg_cost.name,
28     main_program=fluid.default_main_program(),
29     num_trainers=trainer_nums, trainer_id=trainer_id)
30 test_exe = fluid.ParallelExecutor(use_cuda, main_program=test_program,
    share_vars_from=train_exe,
    num_trainers=trainer_nums, trainer_id=trainer_id)
```

(2) 设置环境变量

</>

Python | 收起 ^

```
1 export PADDLE_TRAINER_IPS=172.17.0.2,172.17.0.3
2 export POD_IP=172.17.0.3
3 export PADDLE_TRAINER_ID=0
4 export CUDA_VISIBLE_DEVICES=0
5 export LD_LIBRARY_PATH=/usr/lib/x86_64-linux-gnu/:$LD_LIBRARY_PATH
6 export PADDLE_USE_GPU=1
```

使用时注意：

- (1) 配置 fluid.DistributeTranspilerConfig 中 mode="nccl2" 。
- (2) 调用 transpile 时，trainers 传入所有trainer节点的endpoint，并且传入参数 current_endpoint 。
- (3) 初始化 ParallelExecutor 时传入 num_trainers 和 trainer_id 。

4.3 多机分布式训练checkpoint保存

参数保存

- (1) 基于Trainer 0实现训练过程中的参数保存
- (2) 基于PServer实现了Distribute Lookup Table相关参数保存

使用步骤

- (1) 声明fluid.CheckpointConfig

用户对checkpoint功能的配置，主要是配置对象fluid中的CheckpointConfig
CheckpointConfig 包括4个参数：

1	参数	类型	说明
2	checkpoint_dir	int	checkpoint存储目录
3	max_num_checkpoint	int	最大保存的checkpoint副本数
4	epoch_interval	int	每隔epoch_interval轮epoch
5	step_interval	int	每隔step_interval轮step

(2) 在fluid.Trainer对象的声明中加入fluid.CheckpointConfig的声明
Trainer的init方法的参数中包含了对CheckpointConfig， 需要传入在声明Trainer前声明的CheckpointConfig对象。

核心代码：

</> Python | 收起 ^

```
1 config = CheckpointConfig(  
2     checkpoint_dir = "/tmp/ckpt", max_num_checkpoints = 2,  
3     epoch_interval = 2, step_interval = 10)  
4 trainer = Trainer(..., checkpoint_config=config)
```

定义和声明完成后， 训练在运行过程中就会在指定的step和epoch处进行保存， 出现异常时， 就会自动从最新的checkpoint目录进行参数恢复

注意：

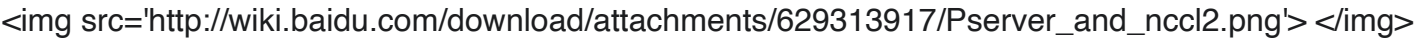
- (1) 保证每个训练的checkpoint_dir 与其他训练独立。
- (2) 最大副本数量max_num_checkpoints需要根据磁盘容量以及模型的大小进行调整， 保证磁盘的可用性。
- (3) epoch_interval 和 step_interval 不宜过小， 频繁的进行checkpoint会拖慢训练速度。
- (4) 分布式训练的过程中： 每个Trainer都会在checkpoint_dir目录中保存当前Trainer的参数（只有Trainer 0会保存模型的参数）， 需要分布式文件系统(HDFS等)将同checkpoint_dir目录的数据进行合并才能得到完整的数据， 恢复训练的时候需要用完整的数据进行恢复。

4.4 Pserver模式和NCCL2模式的对比

4.4.1 Pserver模式和NCCL2模式的区别

Pserver模式中的参数服务器是中心化的， 参数服务器承担了梯度的收集、平均和分发工作。每一次迭代所有的GPU都要与PServer多次通信来获取、更新参数。当trainer节点超过一定数量时， PServer的带宽以及处理能力将成为整个系统的瓶颈。

NCCL2模式采用去中心化的Allreduce方式，这种Allreduce采用的基于环形拓扑的通信方式，在超大规模GPU集群场景下数据通信会有很大的延时开销。



4.4.2 Pserver模式和NCCL2模式的性能对比

由于资源限制，Pserver模式的分布式训练使用两个trainer和一个pserver，每个trainer配置一张GPU卡。NCCL2模式的分布式训练使用两个trainer，每个trainer配置一张GPU卡。

性能分析方法：

使用Python标准库中提供的性能分析工具包，cProfile。

(1) 生产性能分析文件

</> Python | 收起 ^

```
1 python -m cProfile -o profile.out.pserver train_dist_gpu.py
```

(2) 查看性能分析文件

cProfile 在 train_dist_gpu.py 运行完毕后输出profile.out.pserver。可以使用cprofilev来查看性能分析结果。cprofilev是一个Python的第三方库。使用它会开启一个HTTP服务，将性能分析结果以网页的形式展示出来：

1	比较项	NCCL2模式 (percall time)	Pserver模式 (percall time)
2	平均收敛时间	143.66s	317.06s
3	built-in method run	0.040s	0.131s
4	parallel_executor run	0.041s	0.132s
5	built-in method init_devices	10.58s	4.32s
6	cPickle.load	0.082s	0.089s
7	parallel_executor init	0.129s	0.294s
8	get_trainer_program	0s	109.13s
9	transpile	18.121s	60.751s

4.5 Paddle fluid 单机单卡、单机多卡、多机多卡训练的对比

4.5.1 单机单卡、单机多卡、多机多卡的对比

单机单卡：使用普通的执行器Executor，数据一个批次(batch)一个批次训练。

单机多卡：使用并行的执行器ParallelExecutor，一次处理多个批次数据，每个GPU处理一个批次数据计算。变量参数保存在CPU，数据由CPU分发给多个GPU，GPU计算每个批次更新梯度。CPU收

集完多个GPU更新梯度，计算平均梯度，更新参数。继续计算更新梯度。处理速度取决最慢GPU速度。

多机多卡： 使用并行的执行器ParallelExecutor，有单机多卡的特点。处理能力强，适合训练参数超大的模型。

4.5.2 单机训练与分布式训练对比

1	训练模式	训练代码编写	程序运
2	单机训练	直接调用Executor或ParallelExecutor，然后调用run函数	直接运
3	多机训练	设置分布式训练需要的环境变量，通过不同的环境变量来表示不同的节点角色。分布式训练的API，如 DistributeTranspiler。在fluid中，用户只需配置单机训练所需要的网络配置，DistributeTranspiler 模块会自动地根据当前训练节点的角色将用户配置的单机网络配置改写成 pserver 和 trainer 需要的网络配置。结束时，调用exe.close()	通过Ku trainer

4.5.3 单机训练与分布式训练性能对比

1	训练模式	平均一轮训练时间 (s)	mini-batch平均时间 (s)
2	单机两卡	85.46s	0.2s
3	多机单卡	52.88s/44.32	0.20s/0.27s
4	多机两卡	66.43s/53.23s	0.26s/0.32s

5. 集群

5.1 Kubernetes集群运行Paddle分布式训练

在训练之前，将配置与训练数据切分好放在分布式文件系统预先分配好的目录中。训练时，程序从此目录拷贝文件到容器内进行训练，将结果保存到此目录里。

整体的结构图如下：

如上图所示，描述了3个节点的分布式训练场景，在每个Pod上都通过volume方式挂载分布式文件系统的一个目录用于保存训练数据和输出结果。Kubernetes为本次训练创建了3个pod并且调度到3个node上运行。在容器创建之后，会启动Pserver与trainer进程，读取volume中的数据进行这次分布式训练。

在Kubernetes集群上进行Paddle分布式训练，步骤如下：

- 1. 制作Paddle镜像
 - 2) 将训练文件与切分好的数据上传到共享存储
 - 3) 编写本次训练的Yaml文件，创建一个Kubernetes job
- 2. 训练结束后，查看输出结果

5.2 MPI集群运行Paddle分布式训练

在Paddle项目的官方文档中，MPI集群训练采用docker的方式部署，类似MPI on K8S，可以[参考源码目录](#)：[paddle/scripts/cluster_train_v2/openmpi](#)。

使用MPI集群运行Paddle分布式训练的步骤：

(1) 准备Open MPI集群

通过K8S分别启动Open MPI集群的master节点和worker节点，命令如下：

</>Shell | 收起 ^

```
1 kubectl create -f head.yaml
2 kubectl create -f mpi-nodes.yaml
```

然后，通过配置ssh，可以从master节点ssh免密码登陆到Open MPI的每个节点上。

(2) 启动MPI集群作业

手动启动Paddle的MPI集群作业

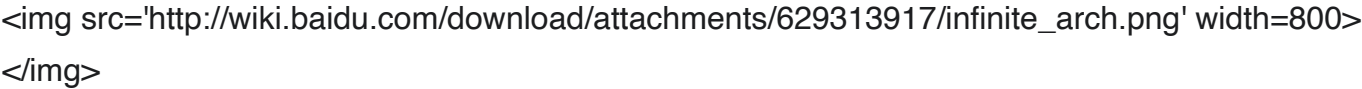
1. 获取master节点和worker节点的IP地址
2. 将worker节点的IP地址保存到machines文件中
3. 拷贝必要的文件到master节点上，如machines、预处理文件、训练文件等
4. 在master节点上准备训练数据、拷贝训练程序和字典文件到每台MPI节点上
5. 在master节点上拷贝训练数据到各自的节点上，启动训练任务

6. Infinite上运行Paddle分布式作业

6.1 创建作业模块功能图



6.2 架构流程图



7. 参考文献

1. [Paddle 官方文档](#)
2. [Paddle Github 源码](#)

